



TrueVision Software Design

Team 3: Yoonseong Kim, Minsu Pak

Brief Summary - Problem

- There are a lot of AI Images
- Sometimes we need to know if
images are AI
- How do tell if an image is AI





Brief Summary - Solution

- Have AI determine if an image is AI
- Put our AI in a web service
- Offer our model as an API



Changes to Requirements

2 Additions and 1 Revision





The Addition

Section 4 - Results Page

Req 4-6: The results page shall allow users to view an AI attention heatmap overlay over the analysed image if they are a paid user

- **Req 4-6.1:** The results page shall allow users to turn off and on the heatmap overlay



The Revision

Req 4-2: The page shall allow users to upload another image to analyze

- **Req 4-2.1:** The page shall allow users to upload an image from their device
- **Req 4-2.2:** The page shall allow users to input a URL to an image

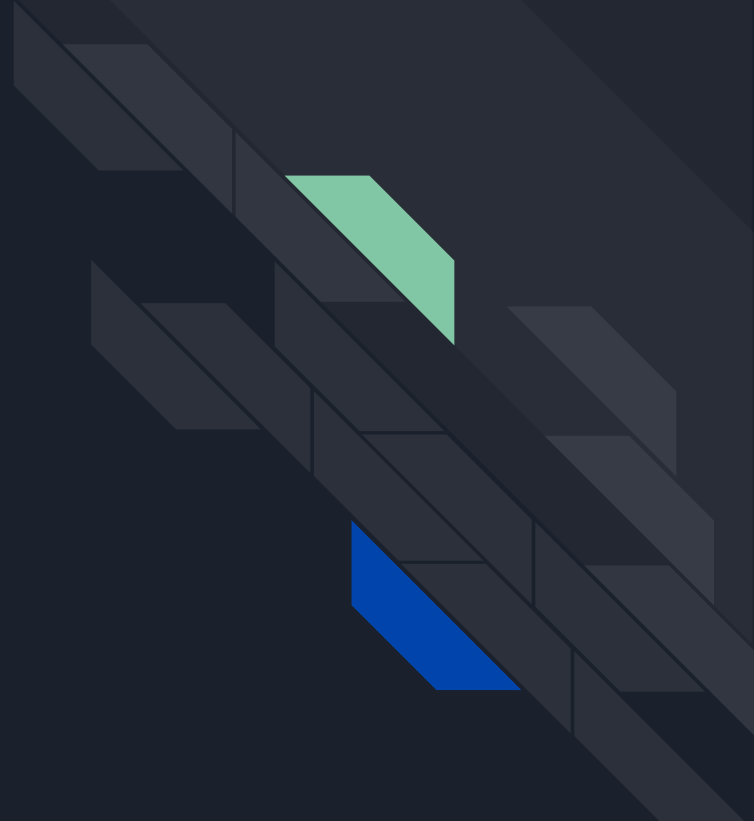
Req 4-3: The page shall allow users to click to confirm their image upload and begin analysis



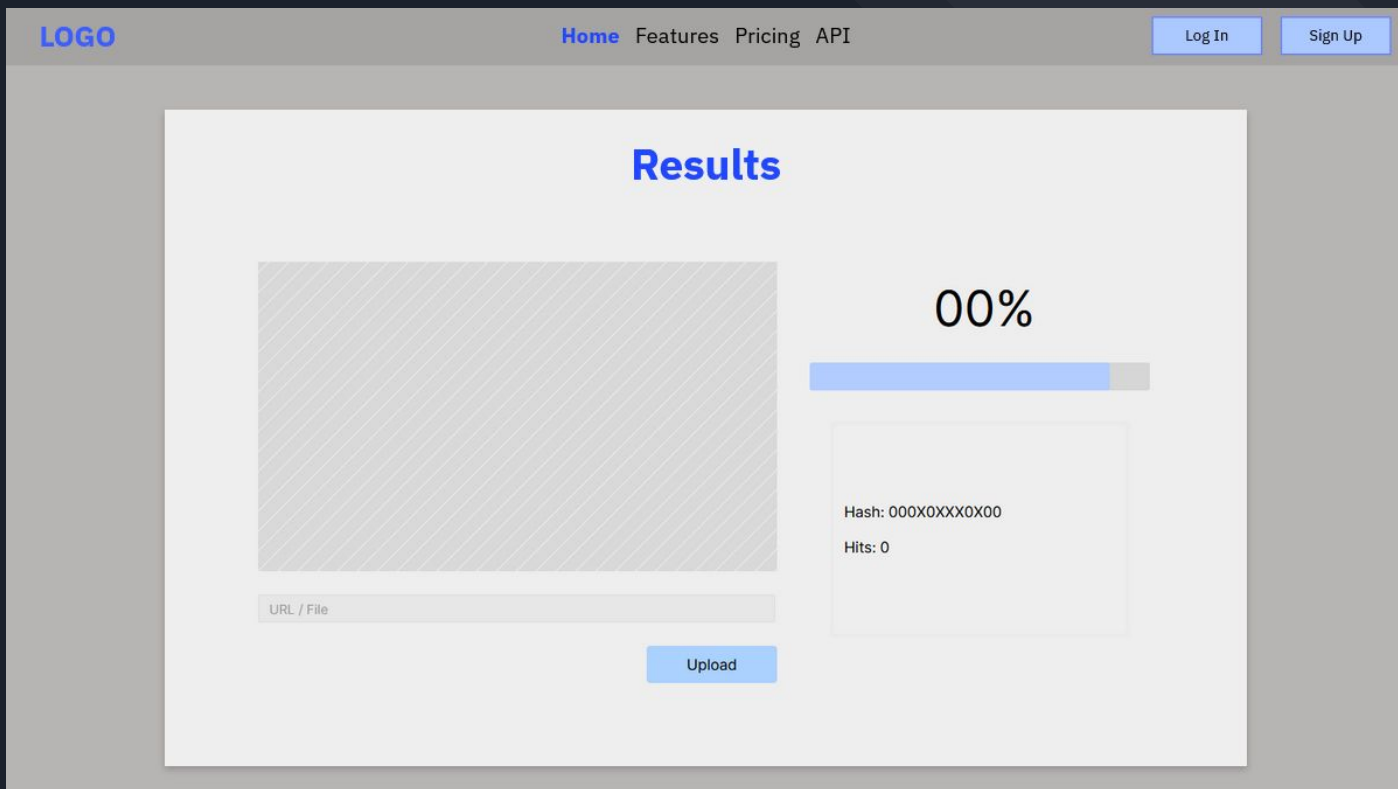
Req 4-2: The page shall link back to the home/upload page

Changes to Design?

Yes



Result Page Previously



Result Page Now

[TrueVision](#)[Analyze](#)[Features](#)[Pricing](#)[API](#)[Log In](#)[Sign Up](#)

Model Used
art

Our analysis says this image is:
Human

93.4%
Confidence

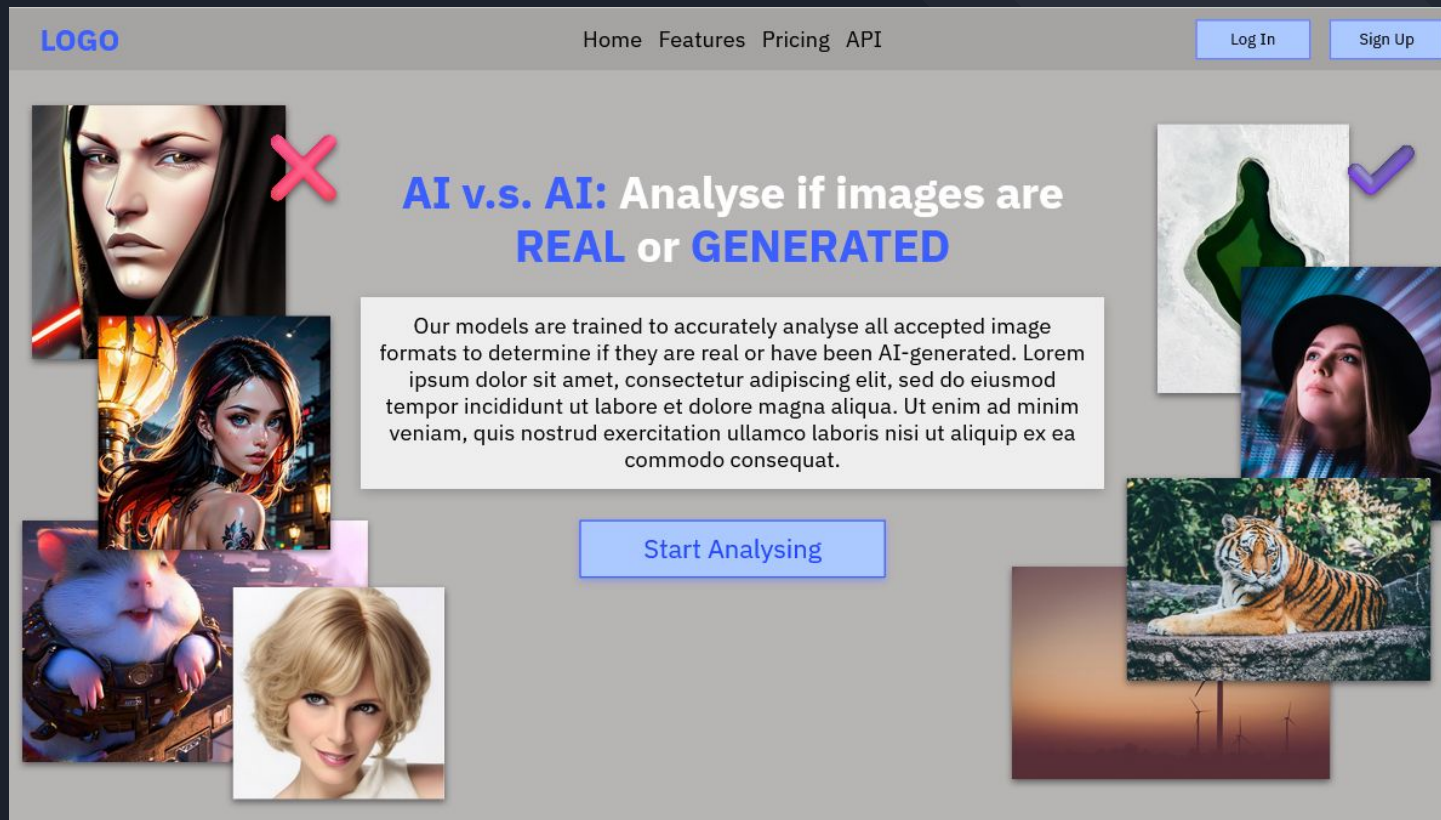
Scan Another Image

Model's Attention

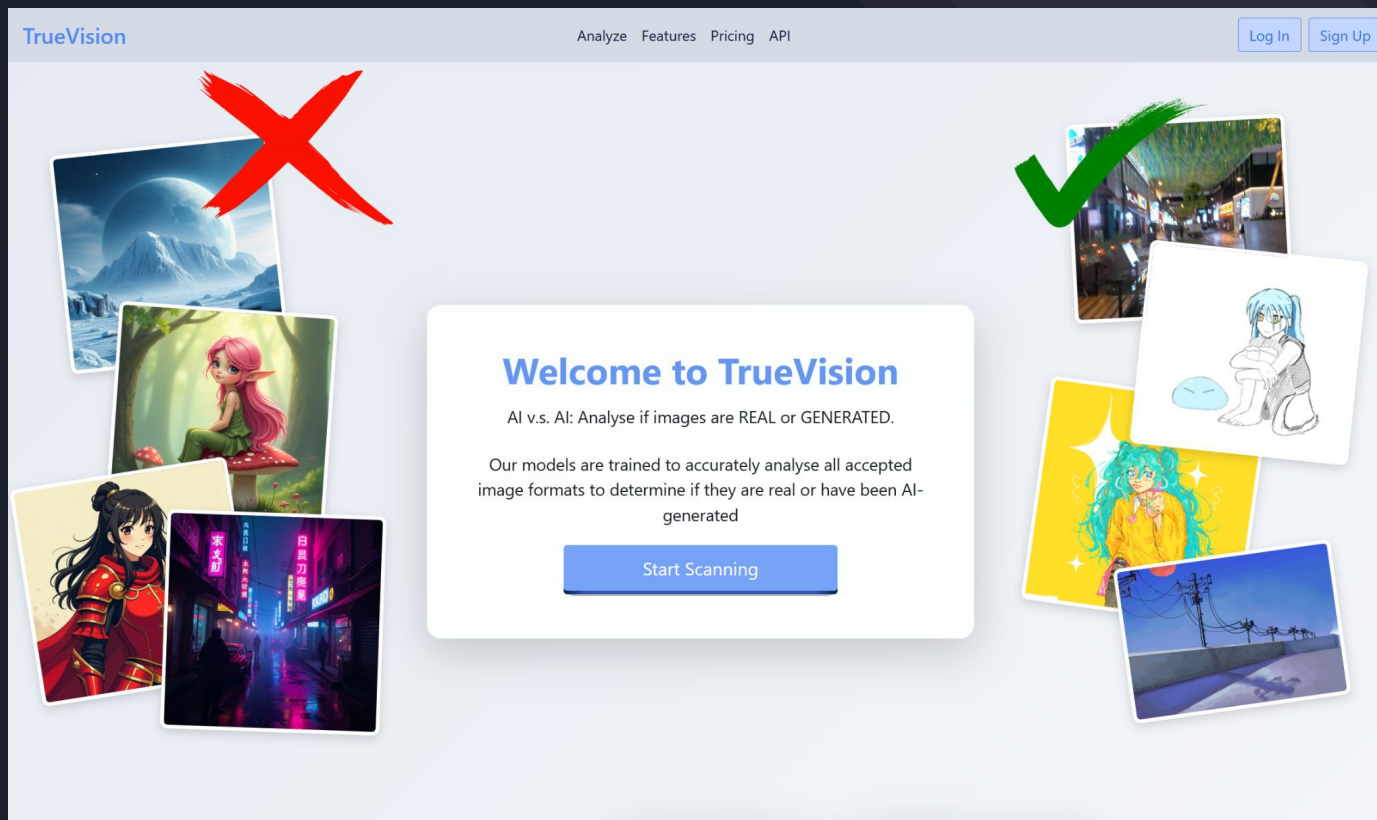


Display Options
☒ Show Overlay
Swap to Mask
When overlay is off, the original image is displayed.

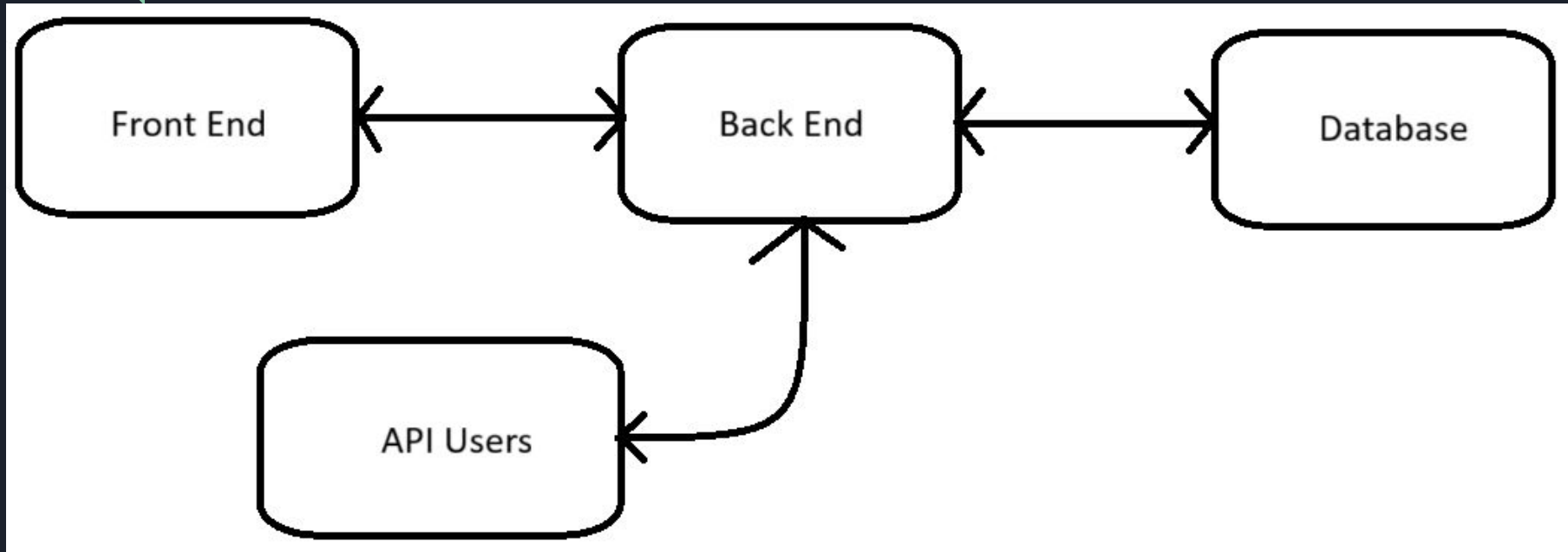
Landing Page Previously



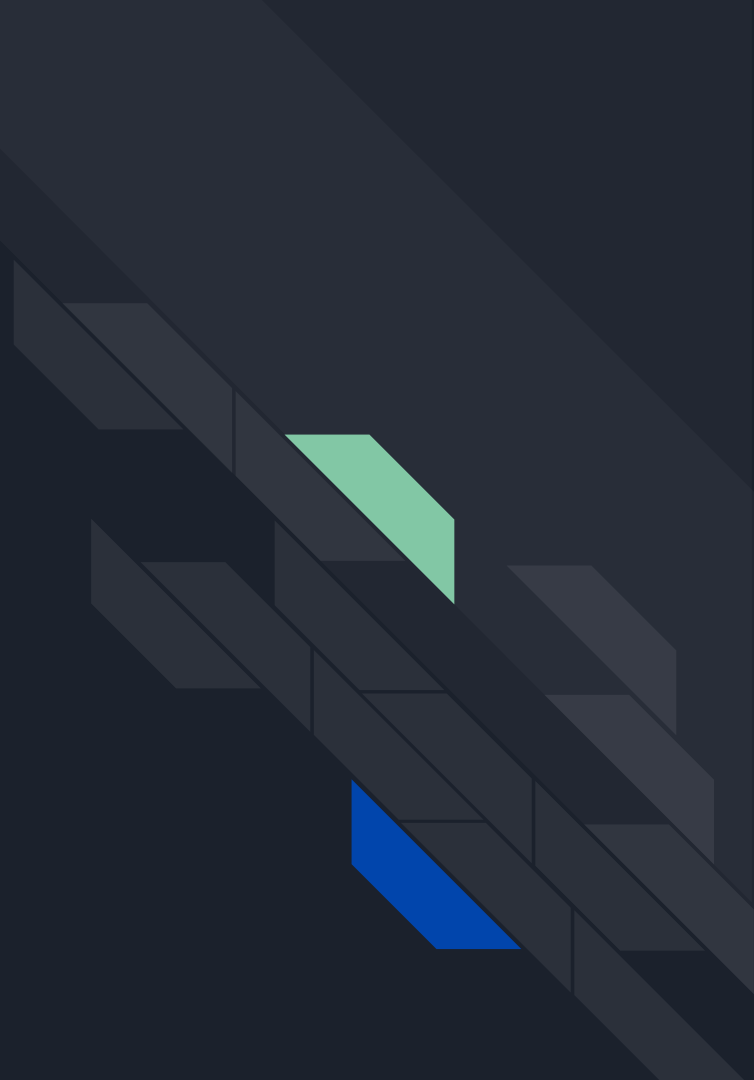
Landing Page Now - no more copyright infringement



Architectural Design



Chosen Technologies





Front End

Chosen Technologies:

- Github Pages
- React + Bootstrap
- Typescript

Reasons:

- Free
- Provides HTTPS
- Streamlines Development
- Catches Errors Early



Back End

Chosen Technologies & Libraries:

- Personal Desktop
- Python
- FastAPI
- Transformers:
vit-base-patch16-224-in21k

Reasons:

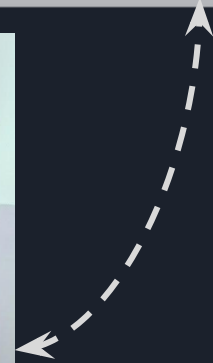
- Free (Technically)
- Python is a simple language
- Possible to Create Custom Models



Back End - Database

Chosen Libraries:

- PyMongo
 - Python driver for MongoDB
 - Compatible with FastAPI and async calls
- PyDantic
 - Automatically converts between BSON (MongoDB) and JSON (backend)
 - Ensures type and collection data validation





Database

Chosen Technologies:

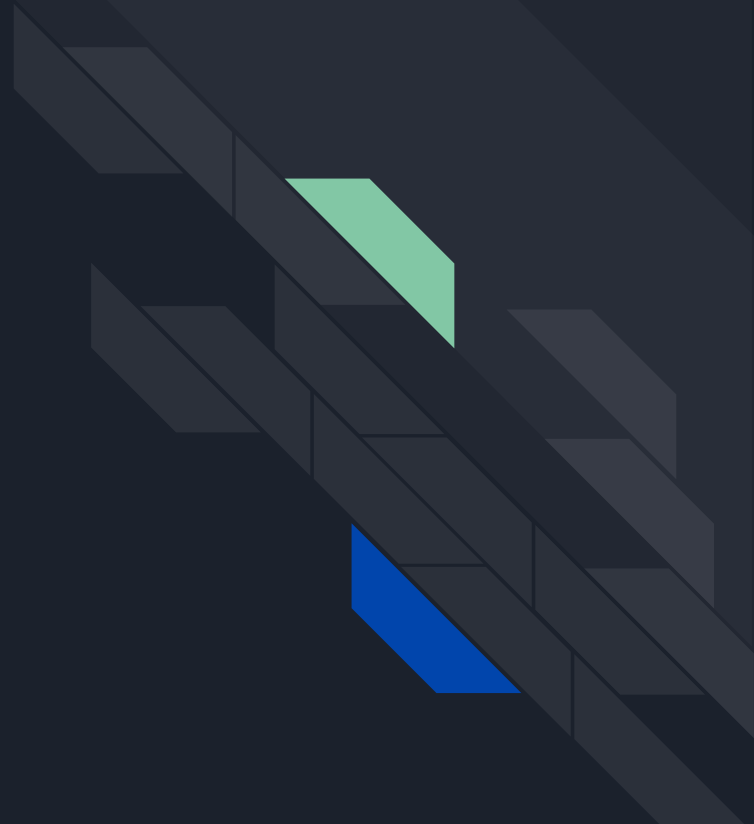
- MongoDB
- MongoDB Atlas cloud service
 - 3 AWS instances

Reasons:

- Free
- Adequate storage and performance
- Flexible schema
- Auto generated ObjectID

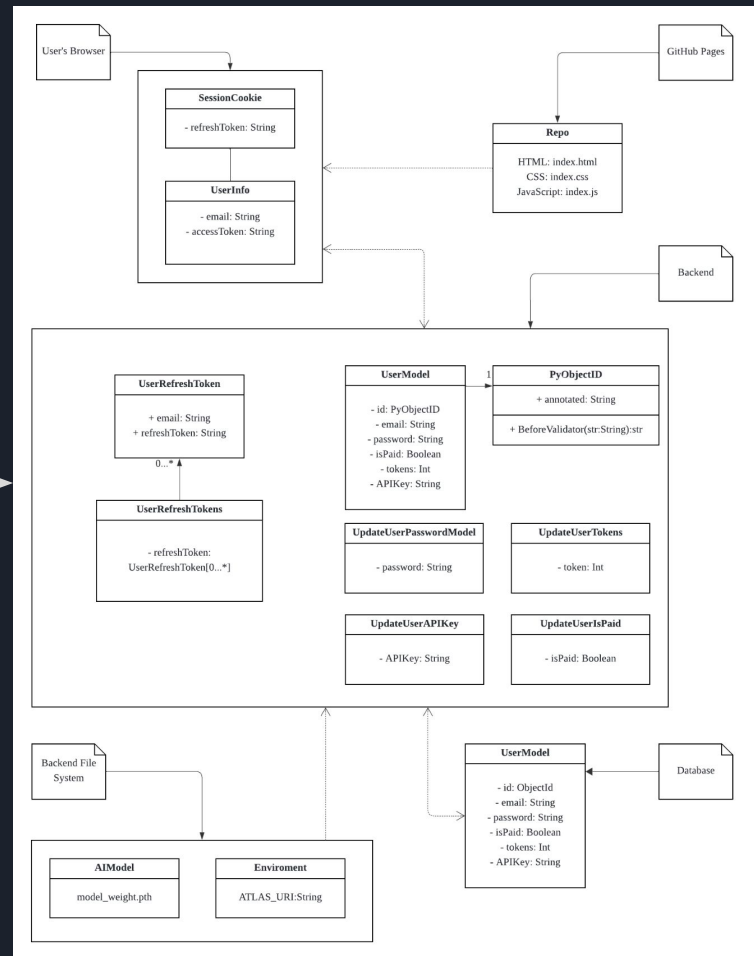
The image shows the AWS logo, which consists of the letters "AWS" in a bold, white, sans-serif font. The logo is positioned on an orange background with horizontal white stripes.

Data Design



Data Design

Reminder





API Design

Name	HTTP Method	URL	Input	Output
Login	POST	/login	String: Email String: HashedPassword	Boolean: Result String: AccessToken Cookie: RefreshToken Status: issues
Sign Up	POST	/sign_up	String: Email String: HashedPassword	Boolean: Result Status: issues
Analyze Image Web	POST	/api/analyze_image_web	String: UserID File: Image	Int: Classification Float: Confidence B64 String: OriginalImage B64 String:HeatMap B64 String: Masked
Analyze Image API	POST	/api/analyze_image_api	String: UserID File: Image String: APIKey	Int: Classification Float: Confidence B64 String: OriginalImage B64 String:HeatMap B64 String: Masked

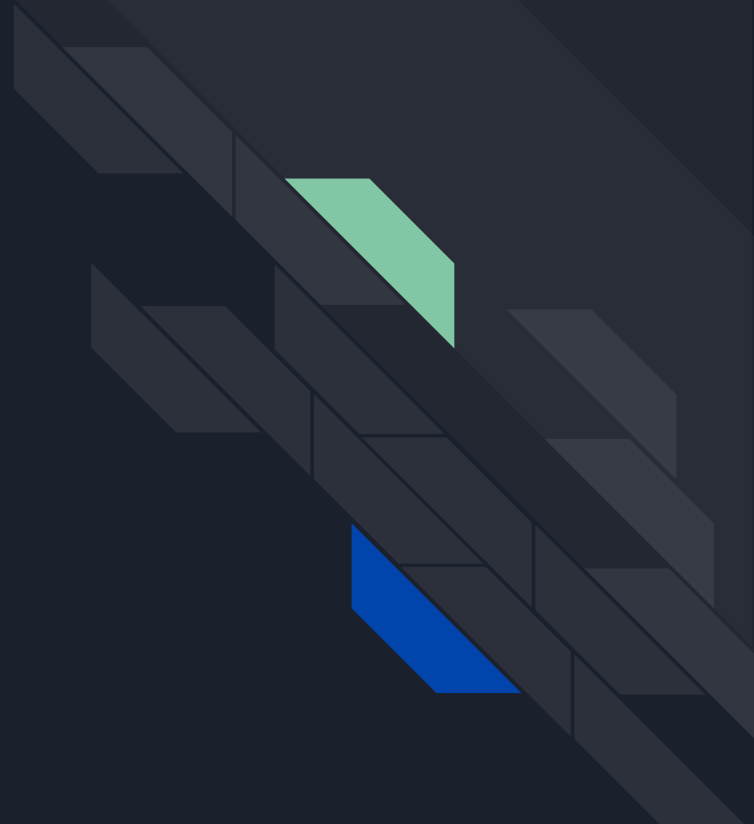
API Design

Name	HTTP Method	URL	Input	Output
Generate API Key	POST	/api/generate_key	String: UserID String: AccessToken	Boolean: Result Status: issues
Change Password	POST	/change_password	String: UserID String: AccessToken String: Old PasswordHash String: New PasswordHash	Status: issues
Purchase Plan	POST	/purchase_plan	String: UserID String: AccessToken	Boolean: Result Status: issues
Purchase Token	POST	/purchase_token	String: UserID String: AccessToken Float: TokenAmount	Boolean: Result Status: issues
Cancel Plan	POST	/cancel_plan	String: UserID String: AccessToken	Boolean: Result Status: issues

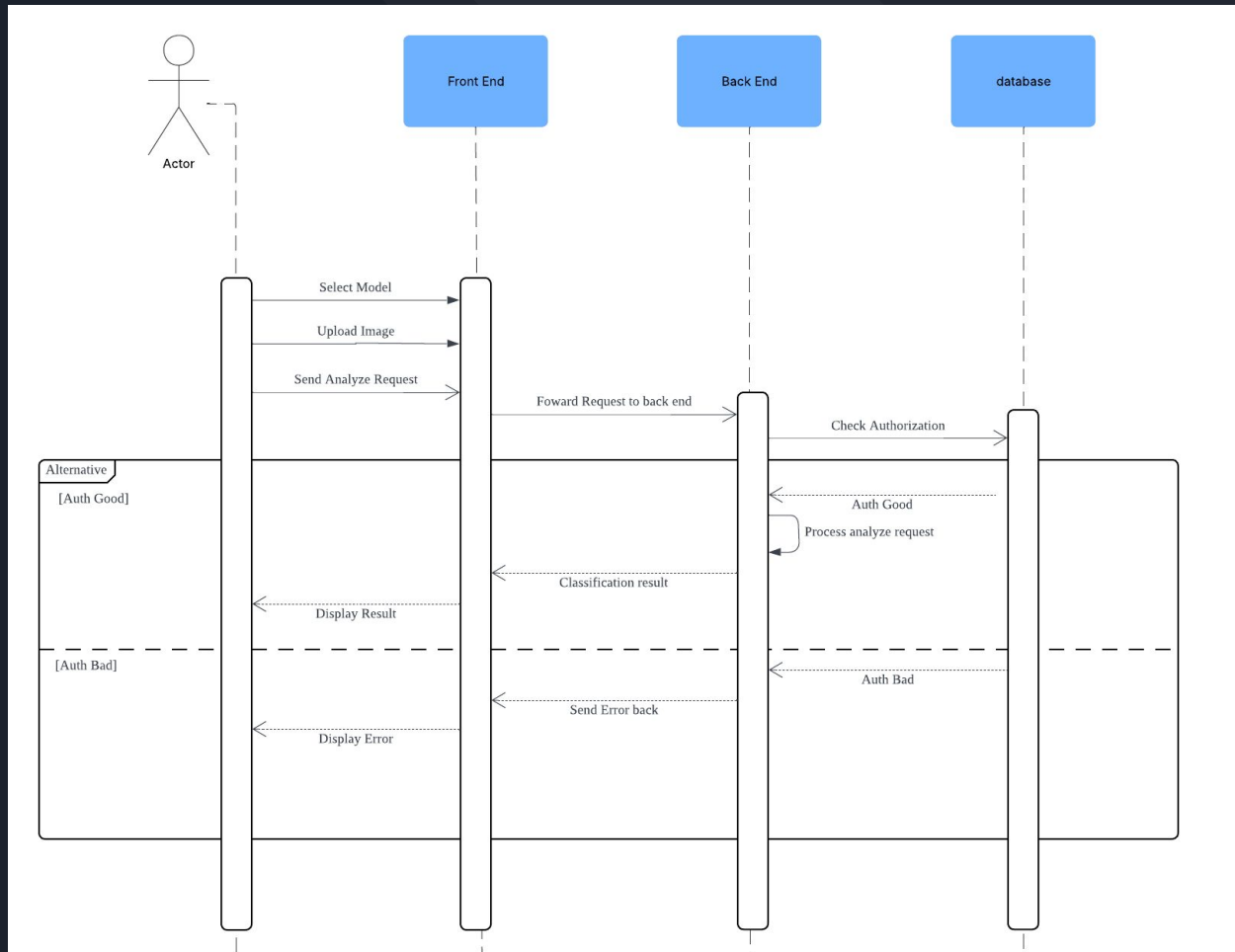
API Design

Name	HTTP Method	URL	Input	Output
Delete Account	POST	/delete_account	String: UserID String: AccessToken	Boolean: Result Status: issues
Refresh User Authorization	GET	/refresh	Cookie: RefreshToken	Status: issues String: AccessToken String: UserID
Log Out	GET	/logout	Cookie: RefreshToken	Status: issues

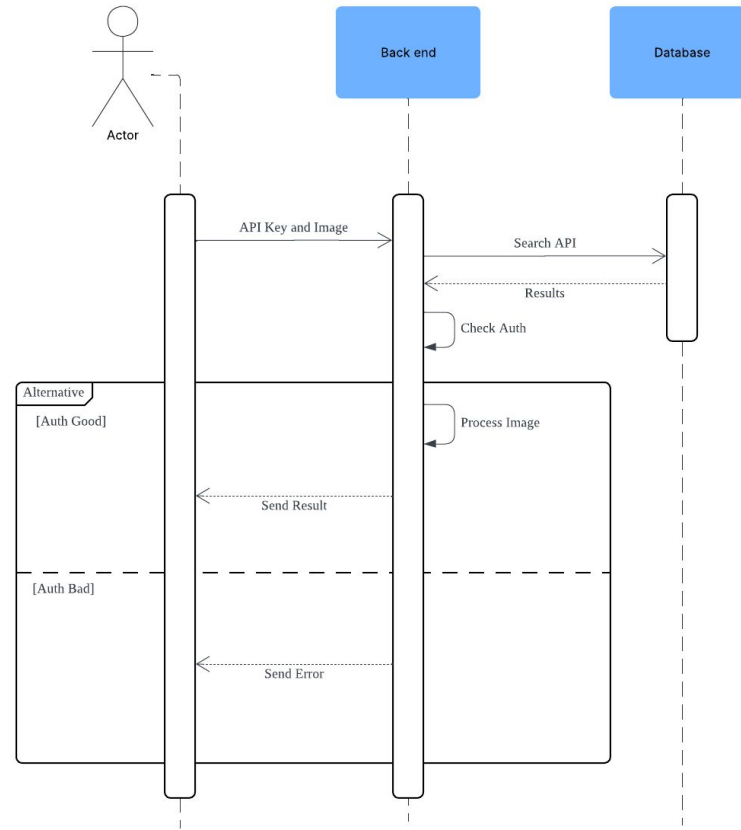
Data Sequence Diagrams



User Uses Analyze Image Feature



Advanced User API Call



Schedule



Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
27	28	29	30 Milestone 1	31	1	2
3	4 Milestone 2	5	6	7	8	9
10	11 Milestone 3	12	13	14	15	16
17	18	19	20 Milestone 4: Beta Release	21	22	23

Milestone 1: 30th October

- Minsu: Frontend API functionality for user account functions, Complete Purchase page and link as required
- Andy: Complete token Purchase page, Adjust frontend API calls on Analyze page

Milestone 2: 4th November

- Minsu: Implement backend APIs and authentication for user account functionality
- Andy: Add key checking to non-frontend Analyze Image API calls, Make API token generator API, Train Anime Art Model

Schedule



Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
27	28	29	30 Milestone 1	31	1	2
3	4 Milestone 2	5	6	7	8	9
10	11 Milestone 3	12	13	14	15	16
17	18	19	20 Milestone 4: Beta Release	21	22	23

Milestone 3: 11th November

- Test and debug frontend, backend, database, and API functionality
- Document and fix issues
- Deploy application for Beta Release

Milestone 4: 20th November

- Get user feedback on our website
- Implement any required changes from feedback and fix any bugs found
- If possible: Implement stretch goals (image hashing, Google auth, email verification, multithreaded backend)

Questions?

